# WALK·IN: robot training data from a phone scan of any room

Oct 14, 2026 · @Someone

## The problem

**Robot training data is slow and expensive to collect, and it only comes from the rooms you can physically get a robot into.**

Robots learn manipulation from demonstrations. Today, each demonstration needs a real robot, a person teleoperating it, and the physical room the robot will work in. To get data in a new kitchen, lab, office or stockroom, you ship a robot and an operator there and record one episode at a time. Datasets end up covering a handful of rooms, so policies trained on them fail in the rooms they will actually work in.

RoboCasa shows the gap in its own numbers: its dataset holds 600+ hours of human demonstrations, against 1,600+ hours made with automated trajectory tools. Automated generation already outscales people; it just isn't tied to real rooms.

**What we solve:** turn a one-minute phone scan of any real room into thousands of robot training demonstrations in that room, with no robot and no teleoperator on site.

## Why it is hard

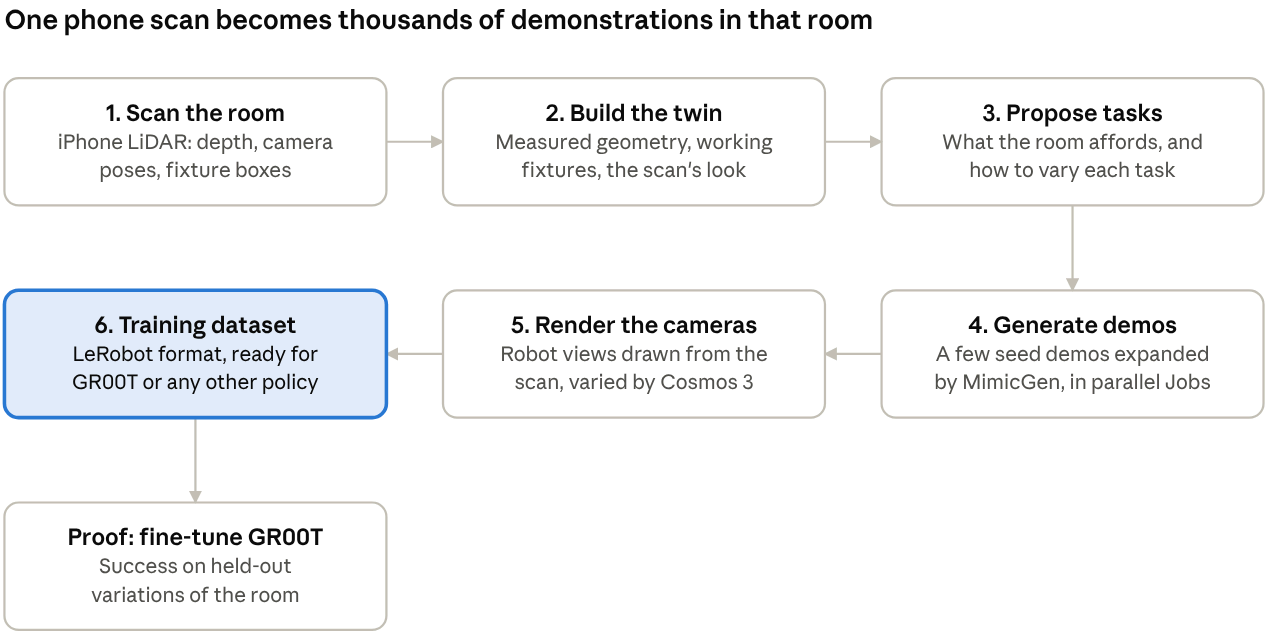
Simulated data is cheap, but it usually fails to transfer. For generated data to be worth training on, it has to be right in four ways at once.

1. **Geometry.** Real dimensions and real layout: counter height, reach distances, what blocks what. Generative reconstruction from photos guesses this; LiDAR measures it.
2. **Appearance.** The robot's camera images must look like the real room. A policy trained on clean simulator renders learns simulator pixels and fails on real ones.
3. **Physics.** Grasps must succeed by contact, and drawers and doors must actually open. WALK·IN today attaches a held object to the hand, which teaches nothing a real robot can use.
4. **Variety.** Thousands of different successful attempts, not one trajectory replayed. That needs object placements, lighting and people that vary, plus an automatic way to solve each variation.

Existing tools each cover part of this. RoboCasa has physics and variety but only in its own synthetic kitchens. Phone scanners capture geometry and appearance but produce nothing a simulator can use. WALK·IN joins them.

## The method

WALK·IN builds a simulator copy of the scanned room that is right in all four ways, then lets the computer collect demonstrations in it instead of a person.




WALK·IN method · 6 steps from scan to dataset

1. **Scan the room.** About a minute with a LiDAR iPhone. Input is a scanner-app export (RGB-D frames with camera poses) plus Apple RoomPlan's room and fixture boxes. Phones without LiDAR use WALK·IN's existing four-photo path.
2. **Build the twin.** Walls and fixture boxes compile into a RoboCasa kitchen layout, which gives real dimensions plus working counters, cabinets, sinks and fridges. Nemotron 3 Nano Omni identifies the objects in the scan and matches them to RoboCasa's 3,200+ object library. The room's appearance comes from a Gaussian splat trained on the scan frames.
3. **Propose tasks.** Nemotron 3 Super reads the twin and lists tasks the room supports, with the variations to try: object, start pose, lighting, people present. Lightning writes varied wordings of each instruction.

4. **Generate demos.** Each task starts from a few seed demonstrations: RoboCasa's human demos for that task type, or about ten teleoperated in the twin. MimicGen re-targets them to new object and fixture poses across hundreds of parallel jobs. Failed attempts are dropped or corrected by WALK·IN's oracle.
5. **Render the cameras.** Each successful episode is re-rendered from the robot's cameras. Static parts of the room come from the splat, so the images carry the real room's pixels; the robot, objects and moving parts come from the simulator, combined by depth. Cosmos 3 Generator restyles a share of frames for lighting and texture variety.
6. **Export the dataset.** GR00T's LeRobot v2 format with `meta/modality.json`, usable by GR00T or any LeRobot policy.

**The key idea is step 5.** Simulator data usually fails because the images look fake. Drawing the static room from the scan itself means the training images show the real room from new viewpoints.

**Scope.** The method works for any scanned room. The demo uses kitchens because RoboCasa's fixture library is kitchen-focused; other rooms use the same path, with plain boxes for fixtures it doesn't recognize. The splat cannot show a drawer opening, so demo tasks are pick-and-place, with drawers as a stretch goal.

## The demo and how we prove it

The video shows one real room becoming a dataset, then a robot trained on that dataset working in the room.

1. Scan a real kitchen with an iPhone, on camera.
2. Split screen: real footage beside the twin, rendered from the same camera path.
3. Hundreds of parallel copies of the room, each with a robot attempting a task, and a counter of demonstrations collected.
4. The dataset exports; GR00T fine-tunes on it; before and after, side by side, in the scanned room.
5. Time and cost per demonstration, generated against teleoperated.

Three measurements back the claim. Tasks: pick an object off the counter and place it in the sink, and move an object between counters.

| Measurement | How | Claim it supports |
|---|---|---|
| Speed and cost | Successful demos per hour and dollars per demo, generated vs. teleoperated. Baseline: we teleoperate the same task in the twin with RoboCasa's teleop tools and time 20 demos | Data is far cheaper and faster |
| Usefulness | GR00T N1.7 success on held-out variations of the scanned room: base model, fine-tuned on generic RoboCasa data, fine-tuned on WALK·IN data. 50 episodes per cell | The data teaches the room |
| Realism | Image and depth error between real scan frames and the twin rendered from the same poses | The data looks like the room |

Usefulness is measured in simulation of the scanned room. Testing on a real robot is the first step after the hackathon.

## Built on Nebius

Data generation is a parallel compute problem, which is why it fits Serverless: one scan fans out into hundreds of short simulation jobs that shut down when done.

| Step | Nebius product | Model or workload |
|---|---|---|
| Read the scan: fixtures, objects, style | Token Factory | Nemotron 3 Nano Omni |
| Assemble the twin and fill occluded gaps | Token Factory | Nemotron 3 Ultra |
| Propose tasks the room affords, and their variations | Token Factory | Nemotron 3 Super |
| Drive simulated people; label each episode | Token Factory | Nemotron 3.5 Lightning |
| Build the twin and its appearance model | Serverless AI Jobs, 1 GPU | One job per scan |

| Step | Nebius product | Model or workload |
|---|---|---|
| Generate and render demonstrations | Serverless AI Jobs, many in parallel | MimicGen and MuJoCo, fanned out |
| Vary lighting and textures in rendered frames | Serverless Endpoint | Cosmos 3 Generator, 1-click template |
| Fine-tune the policy to prove the data works | Serverless AI Jobs, H100 | GR00T N1.7 |
| Store scans, datasets and checkpoints | Object Storage | S3-compatible buckets |
| Host the web app judges use | Serverless Endpoint, CPU | Upload a scan, download a dataset |

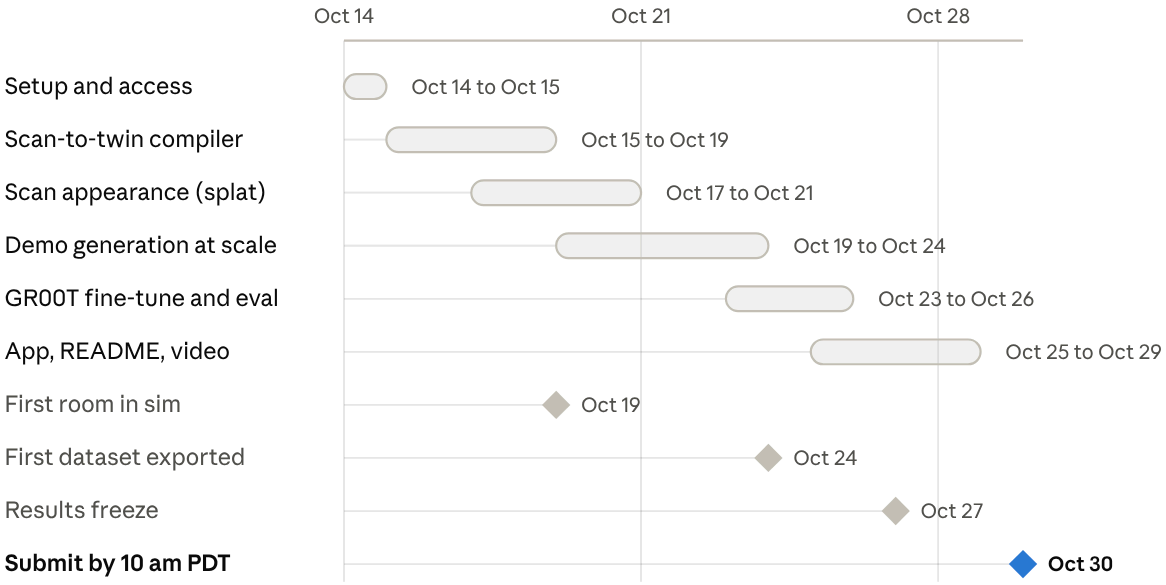
The README reports cost per demonstration split by Nebius product, which doubles as the pricing basis for the product.

## Build plan and risks

The first exported dataset on Oct 24 is the go/no-go point. If it slips, cut the splat and render from simulator textures before cutting the fine-tune.

**First dataset exports Oct 24, and results freeze Oct 27**

Build plan by day, Oct 14 to Oct 30, 2026




Build plan · 6 workstreams, 4 milestones

Submission needs a public repo with an Apache 2.0 license, a hosted demo with pre-scanned rooms, a video under 3 minutes, and a note on what was built during the submission period, since WALK·IN existed before it.

| Risk | Fallback |
|---|---|
| Splat and simulator layers don't line up | Texture the simulator meshes from scan photos instead |
| Scan-to-RoboCasa mapping is messy | Support straight, L-shaped and galley layouts; show three clean scans |
| Gated GR00T backbone access is slow | Request on day 1; train a LeRobot policy with the cookbook recipe meanwhile |
| MimicGen succeeds rarely in new layouts | Teleoperate more seed demos in the twin |
| GR00T gain sits within its 5–6% run-to-run variance | Report it as no gain; the speed and cost results still stand |
| Serverless quota or preemption delays runs | Checkpoint to Object Storage and resume |

## Sources

Opened on 2026-10-14. Model availability changes, so recheck on day 1.

- Isaac GR00T README: N1.7, Apache 2.0, fine-tuning commands, LeRobot data format
- GR00T hardware recommendations: fine-tuning needs 40 GB+ VRAM
- RoboCasa README: RoboCasa365 scope, dataset hours, licenses
- RoboCasa layout example: the layout format the twin compiles to
- RoboCasa MimicGen guide: demo generation for atomic tasks
- Nebius Serverless AI cookbook: Jobs, Endpoints, robotics recipes
- Cosmos 3 Generator template: depth and segmentation transfer controls
- LeRobot fine-tune job: Object Storage pattern for checkpoints
- Token Factory Nemotron models and Nemotron 3 Nano Omni
- WALK·IN: the existing scan-to-sim and data loop

Not verified from here: Apple RoomPlan's output categories and which iPhone scanner apps export depth with camera poses.